

pharo-agentic-browser 概要

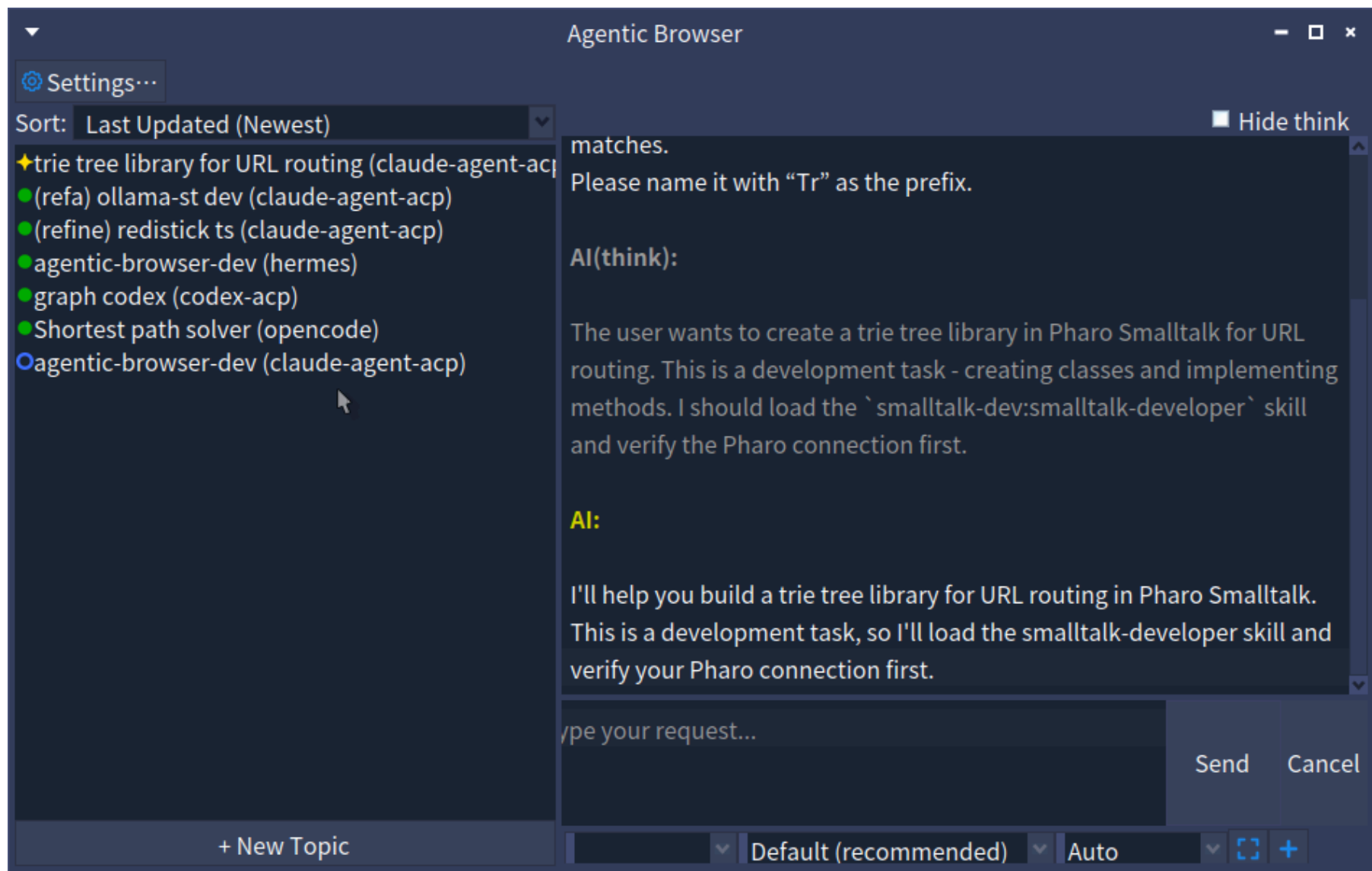
Pharoの統合AIコーディング環境

梅澤 真史

<https://github.com/mumez/pharo-agentic-browser>

pharo-agentic-browser とは？

Claude Code、Codex、OpenCode など、複数のAI コーディングエージェントを
Pharo から利用できる**統合AIコーディングツール**



各AIとのセッションを **トピック** として管理:

1. トピックを作成し、コーディングエージェントを選択
2. リクエストを入力 (`@ClassName` でコード参照、スクリーンショットの添付)
3. AI が自律的に作業 (タスク分解、コード変更、テスト)
4. AI が承認を必要とする場合、チャット内で一時停止して確認
5. プルダウンで応答すると AI が再開
6. トピックの状態は常にサイドバーで確認可能

開発の動機

AI コーディングエージェント専用の GUI ツールが標準に:

- **Claude Desktop** — ツール、MCP、ファイルアクセスを備えた GUI版のClaude
- **Codex App** — OpenAI の自律的なコーディング環境
- **Cursor / Antigravity / Kiro** — AI ネイティブなエディタ

CUIに比べ、AI エージェントとやり取りする際の敷居が下がった
最初は単純なコード補完やチャットだったが...

変化の流れ: チャット → マルチエージェントへの委譲

時代	パラダイム	例
黎明期	エディタサイドバーでのチャット	ChatGPT, GitHub Copilot
少し前	エージェントモードを持つ AI ファーストな IDE	Cursor, Windsurf
現在	タスク全体をエージェントに委譲	Antigravity 2.0, Codex App

ツールは日々進化している

もはや「エディタ + チャット」ではなく、UI は **セッションのオーケストレーション** へと向かっている

なぜ Pharo にもこれが必要か

Pharo 開発者も同じパラダイムを享受すべき:

- リッチなライブ環境 — クラス、メソッド、ランタイムがすぐそこにある
- **pharo-acp** が複数エージェント向けの ACP クライアントをすでに提供
- 複数プロジェクトを1つのイメージ内で並行実行可能

複数の AI エージェントを制御できる Pharo ネイティブなツールが、次のステップとして自然

優位性	詳細
コンテキストを直接渡せる	<code>@ClassName</code> 、 <code>@Class>>method</code> 、スクリーンショット — コピー不要
エージェント非依存	pharo-acp 経由でさまざまなエージェントと連携
Pharo との統合	テスト、コードの変更監視がライブイメージの中で利用可能
並行セッション	複数エージェントを異なるトピックで同時実行

インストール

Pharo 12以上 のイメージで Playground を開き、以下を評価:

```
Metacello new  
  baseline: 'AgenticBrowser';  
  repository: 'github://mumez/pharo-agentic-browser:main/src';  
  load: 'all'.
```

AgenticBrowserを開く:

```
AgenticBrowser open.
```

ACP 対応であれば、どのエージェントでも利用可能:

エージェント	インストール
Claude Code	<code>npm install -g @agentclientprotocol/claude-agent-acp</code>
Codex	<code>npm install -g @agentclientprotocol/codex-acp</code>
Gemini CLI	ACP 組み込み (<code>gemini --acp</code>)
Copilot CLI	ACP 組み込み (<code>copilot --acp --stdio</code>)
Cursor CLI	ACP 組み込み (<code>agent acp</code>)

OpenCode、Kilo Code、Kiro CLI などにも対応

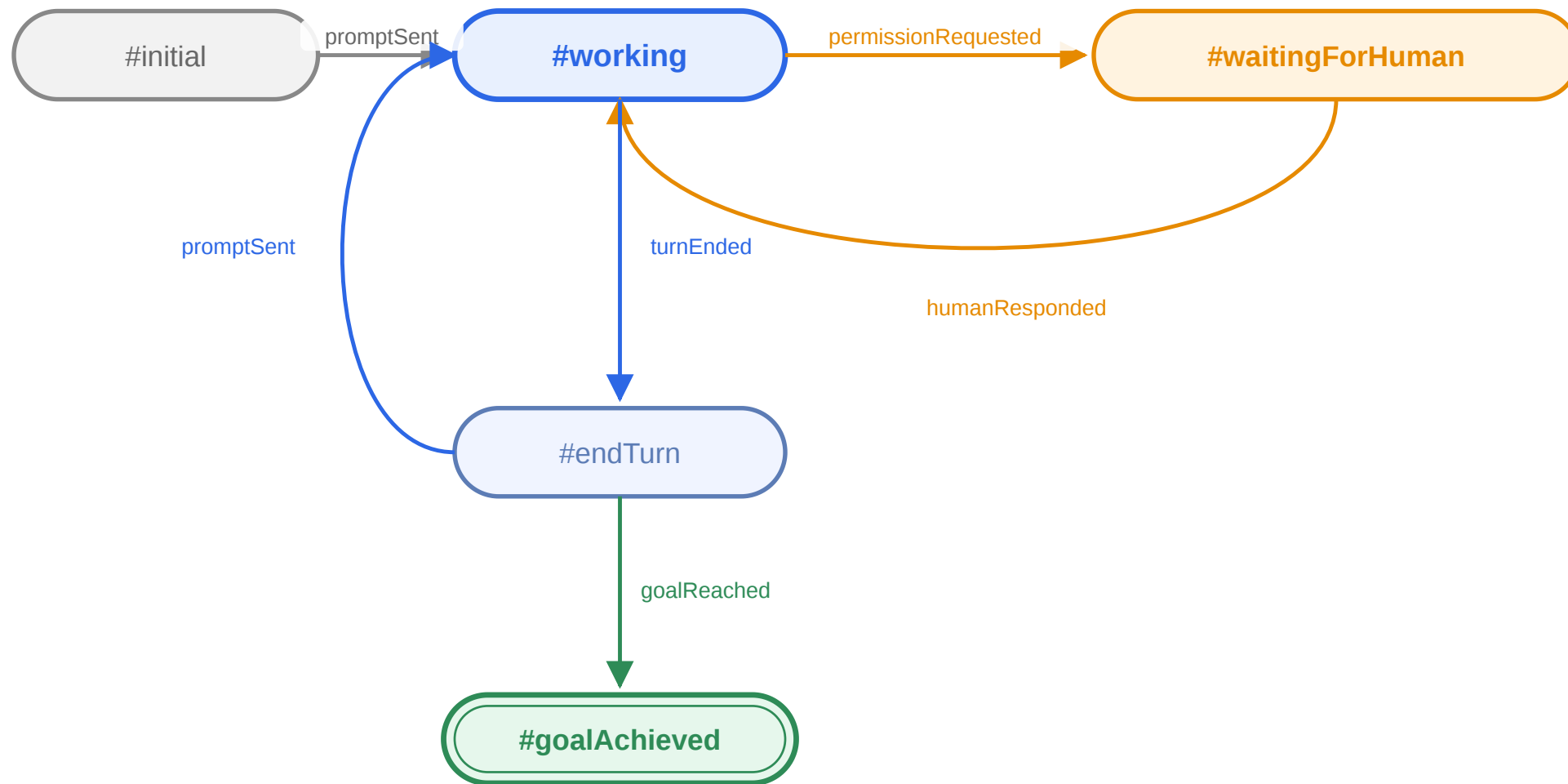
注意: Smalltalk コードの質を上げるため、エージェントには [smalltalk-dev-plugin](#) をインストールしておく

基本機能

1. + **New Topic** をクリック
2. タイトルを入力し、コーディングエージェントを選択
3. (任意) 既存のプロジェクトディレクトリを設定
4. **Create** をクリック — トピックが左サイドバーに表示される
5. (任意) 右クリック → **Set Target Packages...** で追跡対象パッケージを設定

最初のメッセージには、Smalltalk開発者スキルを有効化するため、自動的に
`/st-buddy` が付与される (設定で変更可)

各トピックの状態はステートマシン(SState)で明確に管理:



アイコン	状態	意味
	working	AI が作業中
	waitingForHuman	AI の承認待ち
	endTurn	ターン終了
	goalAchieved	ゴール達成

AI が承認を要求すると:

- ステータスが **?** (waitingForHuman) に変化
- **Send** ボタンが **Allow** に変化
- **Cancel** ボタンが **Deny** に変化

ボタンをクリックして応答すると、AI が再開

承認は会話フローの一部として行われる (モーダルダイアログは使わない)

チャット内で Pharo のクラスやメソッドを直接参照できる:

```
@QueryClass @DBAdapter>>connect please refactor this
```

AgenticBrowser は各 `@mention` を Tonel のソースとして解決し、ACP のテキストリソースにして添付 — コピペ不要。

ドラッグ&ドロップ

- System Browser から クラス をドラッグ → `@ClassName` を挿入
- System Browser から メソッド をドラッグ → `@ClassName>>methodName` を挿入

【 】 ボタンをクリックしてスクリーンショットを添付:

1. ボタンをクリック — カーソルが十字カーソルに変化
2. ドラッグして範囲を選択
3. @sc-20260528-001.png のようなメンションが入力欄に挿入される
4. 送信 — PNG が画像リソースとしてプロンプトに添付される

+ ボタンをクリックしてディスク上の任意のファイルを添付:

1. ボタンをクリック — ファイル選択ダイアログが開く
2. ファイルを選択 — `[filename]` のようなメンションが入力欄に挿入される
3. 送信 — ファイルの内容がテキストリソースとして添付される

サイズの大きいファイルは `maxAttachmentSize` (デフォルト 5MB) に切り詰められる。送信前にメンションテキストを削除すれば添付はキャンセルされる。

トピックを右クリック → **Set Goal...** で完了条件を入力:

```
all tests pass
```

AgenticBrowser は AI にゴール用のプロンプトを送信。

結果報告ファイル(`result-<topic-id>.md`)が作成されると、トピックは ☒ (`#goalAchieved`) に遷移。

ゴール達成時にはアナウンサーやコールバックブロックのフック(`whenGoalAchieved`)も発火するので、独自の自動化ワークフローに統合可能。

トピックに関連するパッケージへの編集をイメージ内で監視:

- トピックを右クリック → **Set Target Packages...** で対象プレフィックスを設定
- 対象のクラス/メソッドがイメージ内で保存されると、ユーザに確認
 - OKなら修正したコードがTonelとしてエクスポートされる
- **追跡対象外**の編集は候補として収集され、後から昇格可能

イメージ内でユーザ自身が変更した内容と、AIが見ている Tonel ソースを同期させる仕組み

トピックは Pharo の **Fuel** シリアライザを使って `ab-topics.fuel` に自動保存
イメージ再起動後も利用可能

手動での保存・復元:

```
AbTopicManager save.  
AbTopicManager load.
```

トピックごとの状態（設定、ステータス、会話）はすべて永続化される

カスタマイズ

新規トピック作成時に、作業ディレクトリを指定しない場合、ディレクトリは、`<agenticBrowserRoot>/topic-template` をもとに生成される:

- デフォルトでは `smalltalk-dev-plugin` 向けに調整された `CLAUDE.md` / `AGENTS.md` がコピーされる
- `.claude`、`.opencode` などのエージェント設定用のディレクトリを配置できる
 - スキル、コマンド、ルールを全トピックで共有
- トピックテンプレート全体を独自にカスタマイズしたものに置き換え可能

新しいトピックごとにコーディングエージェントを再設定する手間を省ける

- MCP サーバー — AgenticBrowser のルートに `mcp.json` を配置。組み込みの `smalltalk-interop` / `smalltalk-validator` は自動マージ (`useDefaultMcpServers: false` で無効化可)
- カスタムエージェント — Playground または `ab-settings.json` から既存メニューにないコーディングエージェントを登録

```
AbSettings default codingAgents: (AbSettings default codingAgents copyWith:
  {'name' -> 'my-agent'.
  'command' -> #('my-agent' '--acp')} asDictionary).
AbSettings save.
```

キー	デフォルト	説明
<code>useDefaultMcpServers</code>	<code>true</code>	組み込みの Smalltalk MCP サーバーをマージ
<code>aiPermissionWaitTimeoutSeconds</code>	<code>1800</code>	人間の承認待ちタイムアウト
<code>aiPermissionTimeoutOption</code>	<code>#reject_once</code>	自動応答: <code>allow_once</code> , <code>allow_always</code> , <code>reject_once</code>

設定は右クリック → **Edit Settings...** からトピックごとにも設定可能

拡張機能

任意の Web ブラウザから **AgenticBrowser** を利用できるオプションパッケージ

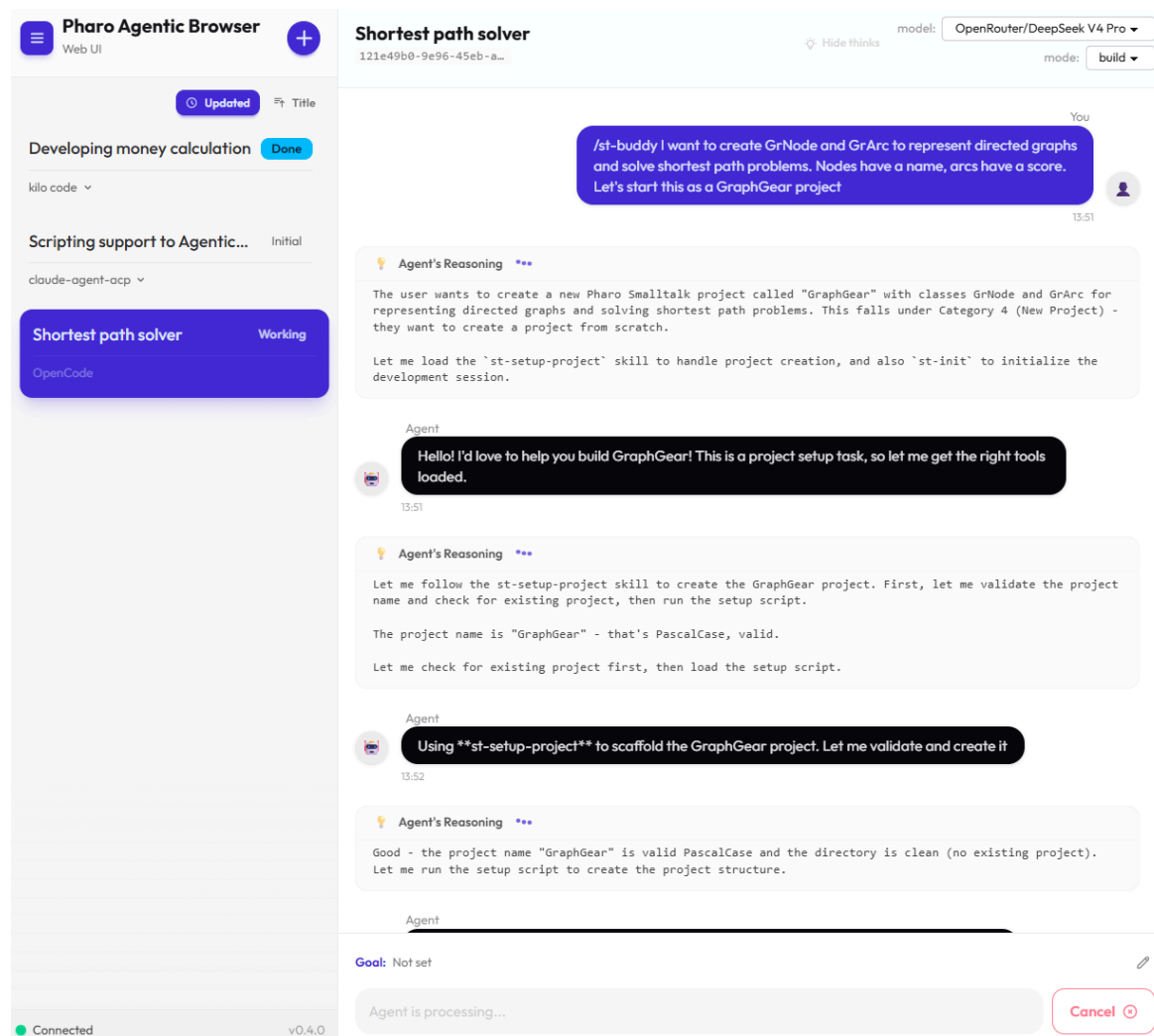
- WebSocket フレームワーク [Ripple](#) により、リロード不要でリアルタイムに同期
- SolidJS + daisyUI でモバイル端末にも対応
- Spec UI 上とほぼ同じ操作（トピック管理、プロンプト送信、承認など）が可能

ユースケース:

外出先からのスマートフォンでの確認

ディスプレイのないヘッドレス環境からの利用

→ 詳細は [Web UI スライド](#) を参照



Smalltalkのスクリプトで AgenticBrowser のトピックをオーケストレーションで
きるオプションパッケージ

- `seq:`、`para:`、`topicBy:`、`agentBy:` などわずか数個のメッセージでマルチエージェントのワークフローを構築
- 結果はステップ間で自動的に受け渡される
 - 手動での情報のやりとりは不要
- AI エージェント自身がスクリプトを書き `st-eval` で実行できる
 - `ab-scripting-feature-dev` スキル

ユースケース:

定型的な AI ワークフローの実行、CIへの組み込み、複雑なマルチエージェント連携

→ 詳細は [Scripting API スライド](#) を参照

逐次実行ステップ (seq:)

— 各トピックの結果は次のトピックのプロンプトに引き継がれる:

```
AgenticBrowser runBy: [ :builder |  
  builder seq: {  
    builder topicBy: [ :t |  
      t prompt: 'List 3 Pharo Smalltalk features in one sentence each.' ].  
    builder topicBy: [ :t |  
      t prompt: 'Summarize the feature list from the previous step in one sentence.' ]  
  } agentBy: [ :a | a claude ] ].
```

並行実行ステップ (`para:`)

— トピックは並列実行され、結果は次のステップのためにまとめられる:

```
AgenticBrowser runBy: [ :builder |  
  builder para: {  
    builder topicBy: [ :t | t prompt: 'List 3 Pharo Smalltalk language features.' ].  
    builder topicBy: [ :t | t prompt: 'List 3 Pharo Smalltalk development tools.' ]  
  } agentBy: [ :a | a claude ] ].
```

`seq:` と `para:` は自由に組み合わせ可能

Arenaパターン: 複数のモデルで並行開発 → 結果をまとめて優れたほうを採用

- 実践例1: *To-Do* アプリ作成スクリプト
- 実践例2: *RediStick*の*TimeSeries*対応スクリプト

まとめ

pharo-agentic-browser は、コーディングエージェントへの委譲というパラダイムを Pharo にもたらす:

- **ネイティブ GUI** — 複数の AI セッションを Pharo から直接管理
- **エージェント非依存** — ACP 対応エージェントであればどれでも利用可能
- **リッチなコンテキスト** — ドラッグ&ドロップでコードメンション、画面キャプチャ
- **Human-in-the-Loop** — 会話の中で承認 (割り込みダイアログなし)
- **ゴール駆動 & 拡張可能** — 完了条件やフックを指定、MCP サーバーやエージェントのカスタマイズが可能
- **Web UI / Scripting API** — ブラウザからの利用や、スクリプトでのオーケストレーションにも対応

フィードバックとコントリビューションをお待ちしています！

<https://github.com/mumez/pharo-agentic-browser>